

AI Model War – DeepSeek vs. ChatGPT

Anirudh Sanjay Mhaske
Department of Computer Science &
Engineering
University at Buffalo
Buffalo, New York, USA
amhaske2@buffalo.edu

Suvidha Vidyasagar Deshpande
Department of Computer Science &
Engineering
University at Buffalo
Buffalo, New York, USA
suvidhav@buffalo.edu

Vidit Rajesh Prabhu
Department of Computer Science &
Engineering
University at Buffalo
Buffalo, New York, USA
viditraj@buffalo.edu

Abstract— This project compares ChatGPT and DeepSeek based on user engagement, retention, and response accuracy to determine their effectiveness. The insights will help optimize AI interactions, improve user experience, and guide future AI advancements.

Keywords— AI Chat Models, DeepSeek, ChatGPT, User Engagement, Retention Rate, Response Accuracy, Churn Rate, AI Performance Evaluation, AI Usability, Data Analysis, AI Adoption Trends

I. INTRODUCTION

With the rise of AI-driven chat models, industries such as customer service, content creation, and virtual assistance have undergone significant transformation. Platforms like ChatGPT and DeepSeek have gained popularity, but their effectiveness varies across accuracy, response speed, and user retention. A comprehensive evaluation of these AI models is essential to determine their real-world applicability and efficiency. This study aims to analyse key performance indicators and provide actionable insights to enhance AI-driven communication.

II. PROBLEM STATEMENT

This project explores the comparative performance of ChatGPT and DeepSeek, focusing on user engagement metrics such as retention rates, response accuracy, churn, and session duration. It seeks to answer key questions: Which AI model retains users better? How do response speed and accuracy differ? What factors influence user churn across platforms? As AI adoption grows, understanding these aspects is essential for optimizing user experience.

Given the dataset's size and complexity, a database is preferred over Excel for efficient querying, scalability, and real-time analysis. This study aims to uncover trends in AI adoption, identify performance gaps, and provide actionable insights to enhance AI-driven interactions

III. BACKGROUND

The rapid evolution of AI chat models has significantly impacted industries worldwide. ChatGPT and DeepSeek are leading conversational AI models, yet their effectiveness varies based on factors like response accuracy, user engagement, and churn rates. High churn rates indicate user dissatisfaction, while strong retention rates suggest a model's effectiveness in sustaining engagement. Given the increasing reliance on AI-powered communication, a data-driven approach is necessary to evaluate these models comprehensively. This study seeks to bridge the gap by providing comparative insights into model efficiency across different user demographics, device types, and interaction patterns.

IV. CONTRIBUTION OF THE PROJECT

This project offers critical insights into AI model performance, helping businesses, researchers, and developers assess user engagement trends and response accuracy. By analysing large-scale user interaction data, it establishes performance benchmarks for ChatGPT and DeepSeek, offers actionable insights for refining AI-driven interactions, and suggests strategies to reduce user churn and enhance retention. These findings are essential for strengthening AI adoption, improving user experience, and advancing AI-driven communication across industries.

V. TARGET USERS

This database will primarily serve AI researchers, data analysts, and product development teams focused on enhancing AI chat models like ChatGPT and DeepSeek. These users will utilize the data to evaluate user engagement, response accuracy, and retention patterns, enabling them to refine AI-driven interactions and improve overall model performance.

In a real-world scenario, a technology company's AI research team could leverage this database to compare model efficiency, track user behaviour trends, and optimize chatbot functionality. The database will be managed by IT professionals or data engineers, ensuring data accuracy, secure access, and seamless processing for large-scale analytics.

VI. ENTITY-RELATIONSHIP DIAGRAM

The Entity-Relationship (E/R) Diagram for our database represents the structure and relationships between key entities involved in analysing AI chat models.

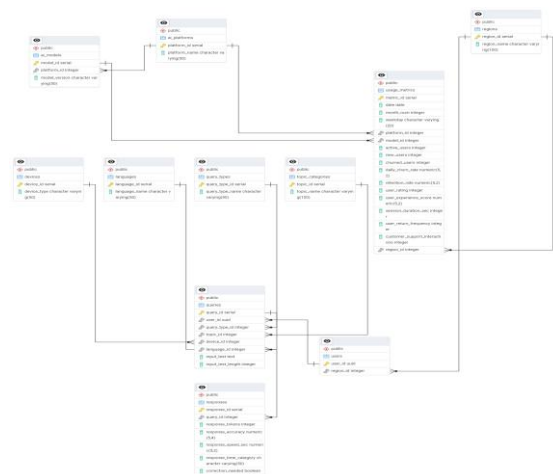


Fig. 1. Entity-Relationship Diagram.

A. users Table

Stores unique users identified by user_id, along with their region, and connects to queries, usage data, and customer support interactions.

B. ai_models Table

Contains AI models identified by model_id, linked to their respective platform_id and model_version, and connects to usage metrics and usage data.

C. ai_platforms Table

Represents AI platforms with platform_id and platform_name, serving as a parent table for AI models.

D. regions Table

Defines regions using region_id and region_name, categorizing users and tracking regional AI usage metrics.

E. query_types Table

Lists different types of queries submitted by users, with query_type_id as the primary key and query_type_name specifying the type.

F. topic_categories Table

Categorizes queries into topics using topic_id and topic_name, helping to analyze AI model interactions.

G. devices Table

Stores device information with device_id and device_type, tracking AI usage across different platforms.

H. languages Table

Maintains supported languages, each defined by language_id and language_name, ensuring multilingual AI interaction tracking.

I. queries Table

Logs user queries with query_id, linking them to user_id, query_type_id, topic_id, device_id, and language_id, while also tracking input_text and input_text_length.

J. responses Table

Stores AI-generated responses tied to query_id, including response_tokens, response_accuracy, response_speed_sec, response_time_category, and whether correction_needed.

K. usage_metrics Table

Captures AI model performance per date, month_num, and weekday, linking to platform_id, model_id, and region_id, while tracking user engagement (active_users, new_users, churned_users), retention statistics (daily_churn_rate, retention_rate), and user feedback (user_rating, user_experience_score).

VII. RELATIONS AND ATTRIBUTES

The Primary Key and Foreign Keys for the relations are:

TABLE I. PRIMARY KEY AND FOREIGN KEYS

KEYS FOR THE RELATION		
I. Table	II. Primary	III. Foreign
ai_platforms	platform_id	-
ai_models	model_id	platform_id
regions	region_id	-

KEYS FOR THE RELATION		
I. Table	II. Primary	III. Foreign
users	user_id	region_id
query_types	query_type_id	-
topic_categories	topic_id	-
devices	device_id	-
languages	language_id	-
queries	query_id	user_id, query_type_id, topic_id, device_id, language_id
responses	response_id	query_id
usage_metrics	metric_id	platform_id, model_id, region_id

Justification for using foreign keys and actions taken when primary key is deleted:

A. ai_models Table

Since multiple models can belong to the same platform, a one-to-many relationship exists between platforms and AI models.

ON DELETE CASCADE : If a platform is deleted, all models associated with it should also be deleted automatically, as they are likely no longer relevant without a platform.

ON UPDATE CASCADE : If the platform_id of a platform changes, the change should propagate to the models that reference it.

B. users Table

Users belong to a specific region, ensuring location-based analysis.

ON DELETE SET NULL : If a region is deleted, users can still exist without a region assigned, but the region_id would be set to NULL to indicate that the user no longer belongs to a region.

ON UPDATE CASCADE: If the region_id changes, it should propagate to the users table to maintain consistency.

C. queries Table

Queries are associated with users and need classification based on topic, device, and language, creating a many-to-one relationship with multiple reference tables.

ON DELETE CASCADE: If a user is deleted, all their associated queries should also be deleted, as they are tied to that specific user.

ON UPDATE CASCADE : If the user_id changes, it should propagate to the queries table to maintain consistency.

ON DELETE SET NULL : If a query type is deleted, the queries still exist but should no longer reference a deleted type. The foreign key should be set to NULL.

ON UPDATE CASCADE : If a query type's ID changes, it should propagate to the queries table.

ON DELETE SET NULL : If a topic category is deleted, the queries may still exist without a specific topic. Therefore, the topic_id should be set to NULL.

ON UPDATE CASCADE : If the topic_id changes, it should be updated across all relevant queries.

ON DELETE SET NULL: If a device is deleted, queries can still exist without it. The device_id should be set to NULL

ON UPDATE CASCADE: If the device_id changes, it should propagate to the queries table.

ON DELETE SET NULL: If a language is deleted, the queries may still exist but no longer need to reference a language. The language_id should be set to NULL

ON UPDATE CASCADE: If the language_id changes, it should propagate to the queries table.

D. *responses Table*

Each response is tied to a specific query, ensuring traceability between inputs and outputs.

ON DELETE CASCADE : If a query is deleted, all associated responses should be deleted as they are tied to the query.

ON UPDATE CASCADE : If the query changes, it should propagate to the response table.

E. *usage_metrics Table*

Since AI performance varies across platforms, models, and regions, this table establishes a many-to-one relationship with these entities.

ON DELETE CASCADE : If a platform is deleted, all associated metrics should also be deleted to maintain consistency.

ON UPDATE CASCADE : If the platform_id changes, it should propagate to the usage_metrics table.

ON DELETE CASCADE : If a platform is deleted, all associated metrics should also be deleted to maintain consistency.

ON UPDATE CASCADE : If the platform_id changes, it should propagate to the usage_metrics table.

ON DELETE CASCADE : If a model is deleted, all associated usage metrics should be deleted as they are tied to the model.

ON UPDATE CASCADE : If the model_id changes, it should propagate to the usage_metrics table.

ON DELETE SET NULL: If a region is deleted, the usage metrics can still exist, but the region_id should be set to NULL to indicate that the metric no longer refers to a region.

ON UPDATE CASCADE : If the region_id changes, it should propagate to the usage_metrics table.

VIII. DETAILED DESCRIPTION OF ATTRIBUTES

A. *ai_platforms Table*

platform_id – Uniquely identifies each AI platform. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

platform_name – Stores the name of the AI platform. Datatype: VARCHAR(50) NOT NULL, Default: None.

B. *ai_models Table*

model_id – Unique identifier for each AI model. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

model_version – Specifies the version of the AI model. Datatype: VARCHAR(50) NOT NULL, Default: None.

C. *regions Table*

region_id – Uniquely identifies a region. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

region_name – Stores the name of the region. Datatype: VARCHAR(100) NOT NULL, Default: None.

D. *users Table*

user_id – Unique identifier for each user. Datatype: UUID PRIMARY KEY, Default: System-generated.

E. *query_types Table*

query_type_id – Unique identifier for categorizing different query types. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

query_type_name – Describes the type of query submitted by users. Datatype: VARCHAR(50) NOT NULL, Default: None.

F. *topic_categories Table*

topic_id – Uniquely identifies a topic category. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

topic_name – Describes the name of the topic under which a query falls. Datatype: VARCHAR(100) NOT NULL, Default: None.

G. *devices Table*

device_id – Uniquely identifies a device type. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

device_type – Specifies the category of device used for submitting queries. Datatype: VARCHAR(50) NOT NULL, Default: None.

H. *languages Table*

language_id – Uniquely identifies a language. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

language_name – Stores the name of the language in which queries are made. Datatype: VARCHAR(50) NOT NULL, Default: None.

I. *queries Table*

query_id – Uniquely identifies each user query. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

input_text – Stores the actual query text input by the user. Datatype: TEXT, Default: NULL.

input_text_length – Records the length of the input text for analysis. Datatype: INT, Default: NULL.

J. *responses Table*

response_id – Unique identifier for each AI-generated response. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

response_tokens – Records the number of tokens generated in the response. Datatype: INT, Default: NULL.

response_accuracy – Measures the accuracy of the AI response. Datatype: DECIMAL(5,4), Default: NULL.

response_speed_sec – Tracks the time taken to generate the response. Datatype: DECIMAL(5,2), Default: NULL.

response_time_category – Categorizes responses based on speed. Datatype: VARCHAR(50), Default: NULL.

correction_needed – Indicates whether the AI response required correction. Datatype: BOOLEAN, Default: NULL.

K. *usage_metrics Table*

metric_id – Unique identifier for each AI performance record. Datatype: SERIAL PRIMARY KEY, Default: System-generated.

date – Stores the date when the metric was recorded. Datatype: DATE, Default: None.

month_num – Represents the month in numeric form (1-12). Datatype: INT, Default: None.

weekday – Stores the name of the day of the week. Datatype: VARCHAR(20), Default: None.

active_users – Counts the number of active users. Datatype: INT, Default: NULL.

new_users – Records the number of newly onboarded users. Datatype: INT, Default: NULL.

churned_users – Captures the number of users who stopped using the platform. Datatype: INT, Default: NULL.

daily_churn_rate – Measures the daily churn rate percentage. Datatype: DECIMAL(5,2), Default: NULL.

retention_rate – Tracks the percentage of retained users. Datatype: DECIMAL(5,2), Default: NULL.

user_rating – Stores user ratings for the AI model. Datatype: INT, Default: NULL.

user_experience_score – Captures the overall user experience score. Datatype: DECIMAL(5,2), Default: NULL.

session_duration_sec – Measures the average session duration in seconds. Datatype: INT, Default: NULL.

user_return_frequency – Tracks how often users return to use the AI model. Datatype: INT, Default: NULL.

customer_support_interactions – Counts the number of customer support interactions. Datatype: INT, Default: NULL.

IX. PROOF OF BOYCE-CODD NORMAL FORM

All the given relations are already in BCNF. Below is the proof of the same.

A. *ai_platforms Table*

Functional Dependencies:

platform_id → platform_name

Candidate Key: platform_id (Primary Key)

The table is in BCNF because the only functional dependency has platform_id (the primary key) on the left-hand side.

B. *ai_models Table*

Functional Dependencies:

model_id → platform_id, model_version

Candidate Key: model_id (Primary Key)

The table is in BCNF because model_id is the primary key, and all attributes are functionally dependent on it.

C. *regions Table*

Functional Dependencies:

region_id → region_name

Candidate Key: region_id (Primary Key)

The table is in BCNF because region_id is the primary key.

D. *users Table*

Functional Dependencies:

user_id → region_id

Candidate Key: user_id (Primary Key)

The table is in BCNF because user_id is the primary key, and the dependency is fully functional.

E. *query_types Table*

Functional Dependencies:

query_type_id → query_type_name

Candidate Key: query_type_id (Primary Key)

The table is in BCNF because query_type_id is the primary key.

F. *topic_categories Table*

Functional Dependencies:

topic_id → topic_name

Candidate Key: topic_id (Primary Key)

The table is in BCNF because topic_id is the primary key.

G. devices Table

Functional Dependencies:

device_id → device_type

Candidate Key: device_id (Primary Key)

The table is in BCNF because device_id is the primary key.

H. languages Table

Functional Dependencies:

language_id → language_name

Candidate Key: language_id (Primary Key)

The table is in BCNF because language_id is the primary key.

I. queries Table

Functional Dependencies:

query_id → user_id, query_type_id, topic_id, device_id, language_id, input_text, input_text_length

Candidate Key: query_id (Primary Key)

The table is in BCNF because query_id is the primary key.

J. responses Table

Functional Dependencies:

response_id → query_id, response_tokens, response_accuracy, response_speed_sec, response_time_category, correction_needed

Candidate Key: response_id (Primary Key)

The table is in BCNF because response_id is the primary key.

K. usage_metrics Table

Functional Dependencies:

metric_id → date, month_num, weekday, platform_id, model_id, active_users, new_users, churned_users, daily_churn_rate, retention_rate, user_rating, user_experience_score, session_duration_sec, user_return_frequency, customer_support_interactions, region_id

Candidate Key: metric_id (Primary Key)

The table is in BCNF because metric_id is the primary key.

X. SQL QUERIES

Below are the before and after results of the queries implemented to test the database.

A. Inserting 'French' language

Before:

	language_id [PK] integer	language_name character varying (50)
1	1	English
2	2	Spanish
3	3	German

After:

	language_id [PK] integer	language_name character varying (50)
1	1	English
2	2	Spanish
3	3	German
4	5	French

B. Updating topic 'Health' to 'Healthcare'

Before:

	topic_id [PK] integer	topic_name character varying (100)
1	1	Science
2	2	Technology
3	3	Health

After:

	topic_id [PK] integer	topic_name character varying (100)
1	1	Science
2	2	Technology
3	3	Healthcare

C. Deleting 'Watch' device

Before:

	device_id [PK] integer	device_type character varying (50)
1	1	Mobile
2	2	Desktop
3	3	Tablet
4	4	Watch

After:

	device_id [PK] integer	device_type character varying (50)
1	1	Mobile
2	2	Desktop
3	3	Tablet

D. Join Query on queries, users, regions, devices and languages table

	query_id integer	user_id uuid	region_name character varying (100)	device_type character varying (50)	language_name character varying (50)
1	1	6f264298-fd8d-4509-8ca1-964e6d5cacc6	North America	Mobile	English
2	2	a4fe4796-7dac-475d-89f6-511495aabe44	Europe	Desktop	Spanish

E. Group By Query on queries and topic_categories table

	topic_name character varying (100)	query_count bigint
1	Technology	1
2	Science	1

F. Order By Query on responses and queries table

	response_id integer	response_accuracy numeric (5,4)	input_text text
1	1	0.9423	What is quantum computing?
2	2	0.8756	Summarize this article on AI ethics.

G. Subquery on regions and users table

	user_id [PK] uuid
1	6f264298-fd8d-4509-8ca1-964e6d5cacc6

H. Join and Group By Query on usage_metrics and regions table

	region_name character varying (100)	avg_experience numeric
1	North America	82.5000000000000000
2	Europe	88.0000000000000000

I. Join and Subquery on usage_metrics and ai_platforms table

	platform_name character varying (50)	total_active bigint
1	DeepSeek	1200

J. Inserting 'Smartwatch' device using Procedure

Before:

	device_id [PK] integer	device_type character varying (50)
1	1	Mobile
2	2	Desktop
3	3	Tablet

After:

	device_id [PK] integer	device_type character varying (50)
1	1	Mobile
2	2	Desktop
3	3	Tablet
4	5	Smartwatch

K. Updating 'Technology' topic to 'Tech' using Procedure

Before:

	topic_id [PK] integer	topic_name character varying (100)
1	1	Science
2	2	Technology
3	3	Healthcare

After:

	topic_id [PK] integer	topic_name character varying (100)
1	1	Science
2	3	Healthcare
3	2	Tech

XI. INDEXING AND QUERY EXECUTION ANALYSIS

Below are the before and after results of the indexing the problematic queries.

A. Retrieving query details using queries, users, regions, devices and languages table

The query retrieves query details by performing joins across five tables: queries, users, regions, devices, and languages to display query ID, user ID, region name, device type, and language name.

	QUERY PLAN text	
1	Hash Join (cost=109.75..136.43 rows=810 width=474) (actual time=0.071..0.073 rows=2 loops=1)	
2	Hash Cond: (q.language_id = l.language_id)	
3	-> Hash Join (cost=87.60..112.13 rows=810 width=360) (actual time=0.053..0.054 rows=2 loops=1)	
4	Hash Cond: (q.device_id = d.device_id)	
5	-> Hash Join (cost=65.45..87.84 rows=810 width=246) (actual time=0.037..0.038 rows=2 loops=1)	
6	Hash Cond: (u.region_id = r.region_id)	
7	-> Hash Join (cost=48.25..68.48 rows=810 width=32) (actual time=0.022..0.023 rows=2 loops=1)	
8	Hash Cond: (q.user_id = u.user_id)	
9	-> Seq Scan on queries q (cost=0.00..18.10 rows=810 width=28) (actual time=0.006..0.006 rows=2 loops=1)	
10	-> Hash (cost=27.00..27.00 rows=1700 width=20) (actual time=0.009..0.009 rows=2 loops=1)	
11	Buckets: 2048 Batches: 1 Memory Usage: 17kB	
12	-> Seq Scan on users u (cost=0.00..27.00 rows=1700 width=20) (actual time=0.005..0.005 rows=2 loops=1)	
13	-> Hash (cost=13.20..13.20 rows=320 width=222) (actual time=0.008..0.009 rows=2 loops=1)	
14	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
15	-> Seq Scan on regions r (cost=0.00..13.20 rows=320 width=222) (actual time=0.005..0.005 rows=2 loops=1)	
16	-> Hash (cost=15.40..15.40 rows=540 width=122) (actual time=0.010..0.010 rows=4 loops=1)	
17	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
18	-> Seq Scan on devices d (cost=0.00..15.40 rows=540 width=122) (actual time=0.007..0.007 rows=2 loops=1)	
19	-> Hash (cost=15.40..15.40 rows=540 width=122) (actual time=0.013..0.013 rows=4 loops=1)	
20	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
21	-> Seq Scan on languages l (cost=0.00..15.40 rows=540 width=122) (actual time=0.009..0.009 rows=2 loops=1)	
22	Planning Time: 0.675 ms	
23	Execution Time: 0.111 ms	

Indexes were created on queries.user_id, queries.device_id, queries.language_id, and users.region_id to optimize the join operations.

	QUERY PLAN text	
1	Nested Loop (cost=1.34..49.28 rows=2 width=474) (actual time=0.056..0.061 rows=2 loops=1)	
2	-> Nested Loop (cost=1.19..32.91 rows=2 width=360) (actual time=0.049..0.053 rows=2 loops=1)	
3	-> Nested Loop (cost=1.04..16.55 rows=2 width=246) (actual time=0.038..0.041 rows=2 loops=1)	
4	Join Filter: (q.user_id = u.user_id)	
5	Rows Removed by Join Filter: 2	
6	-> Hash Join (cost=1.04..15.46 rows=2 width=234) (actual time=0.028..0.029 rows=2 loops=1)	
7	Hash Cond: (r.region_id = l.region_id)	
8	-> Seq Scan on regions r (cost=0.00..13.20 rows=320 width=222) (actual time=0.010..0.010 rows=2 loops=1)	
9	-> Hash (cost=1.02..1.02 rows=2 width=20) (actual time=0.009..0.009 rows=2 loops=1)	
10	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
11	-> Seq Scan on users u (cost=0.00..1.02 rows=2 width=20) (actual time=0.004..0.005 rows=2 loops=1)	
12	-> Materialize (cost=0.00..1.03 rows=2 width=28) (actual time=0.005..0.005 rows=2 loops=2)	
13	-> Seq Scan on queries q (cost=0.00..1.02 rows=2 width=28) (actual time=0.005..0.006 rows=2 loops=1)	
14	-> Index Scan using devices_pkey on devices d (cost=0.15..8.17 rows=1 width=122) (actual time=0.005..0.005 rows=1 loops=1)	
15	Index Cond: (device_id = q.device_id)	
16	-> Index Scan using languages_pkey on languages l (cost=0.15..8.17 rows=1 width=122) (actual time=0.004..0.004 rows=1 loops=1)	
17	Index Cond: (language_id = q.language_id)	
18	Planning Time: 0.617 ms	
19	Execution Time: 0.094 ms	

Before indexing, the query had a cost of 109.75..136.43 and an execution time of 0.111 ms. After applying the indexes, the cost reduced to 1.34..49.28 and the execution time improved to 0.094 ms.

B. Calculating average response accuracy using responses, queries and topic_categories table

The query calculates the average response accuracy grouped by topic name by joining the responses, queries, and topic_categories tables.

	QUERY PLAN text	
1	HashAggregate (cost=33.78..36.28 rows=200 width=250) (actual time=0.079..0.082 rows=2 loops=1)	
2	Group Key: t.topic_name	
3	Batches: 1 Memory Usage: 40kB	
4	-> Hash Join (cost=15.49..31.58 rows=440 width=230) (actual time=0.065..0.068 rows=2 loops=1)	
5	Hash Cond: (r.query_id = q.query_id)	
6	-> Seq Scan on responses r (cost=0.00..14.40 rows=440 width=16) (actual time=0.016..0.016 rows=2 loops=1)	
7	-> Hash (cost=15.46..15.46 rows=2 width=222) (actual time=0.036..0.038 rows=2 loops=1)	
8	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
9	-> Hash Join (cost=1.04..15.46 rows=2 width=222) (actual time=0.033..0.034 rows=2 loops=1)	
10	Hash Cond: (t.topic_id = q.topic_id)	
11	-> Seq Scan on topic_categories t (cost=0.00..13.20 rows=320 width=222) (actual time=0.009..0.009 rows=2 loops=1)	
12	-> Hash (cost=1.02..1.02 rows=2 width=8) (actual time=0.013..0.013 rows=2 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on queries q (cost=0.00..1.02 rows=2 width=8) (actual time=0.008..0.008 rows=2 loops=1)	
15	Planning Time: 0.603 ms	
16	Execution Time: 0.145 ms	

Indexes were added on responses.query_id and queries.topic_id to optimize join performance and reduce scan costs.

	QUERY PLAN text	
1	GroupAggregate (cost=16.56..16.60 rows=2 width=250) (actual time=0.105..0.107 rows=2 loops=1)	
2	Group Key: t.topic_name	
3	-> Sort (cost=16.56..16.57 rows=2 width=230) (actual time=0.094..0.095 rows=2 loops=1)	
4	Sort Key: t.topic_name	
5	Sort Method: quicksort Memory: 25kB	
6	-> Nested Loop (cost=1.04..15.46 rows=2 width=230) (actual time=0.040..0.043 rows=2 loops=1)	
7	Join Filter: (r.query_id = q.query_id)	
8	Rows Removed by Join Filter: 2	
9	-> Hash Join (cost=1.04..15.46 rows=2 width=222) (actual time=0.031..0.033 rows=2 loops=1)	
10	Hash Cond: (t.topic_id = q.topic_id)	
11	-> Seq Scan on topic_categories t (cost=0.00..13.20 rows=320 width=222) (actual time=0.011..0.012 rows=2 loops=1)	
12	-> Hash (cost=1.02..1.02 rows=2 width=8) (actual time=0.011..0.012 rows=2 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on queries q (cost=0.00..1.02 rows=2 width=8) (actual time=0.007..0.008 rows=2 loops=1)	
15	-> Materialize (cost=0.00..1.03 rows=2 width=16) (actual time=0.004..0.004 rows=2 loops=2)	
16	-> Seq Scan on responses r (cost=0.00..1.02 rows=2 width=16) (actual time=0.005..0.005 rows=2 loops=1)	
17	Planning Time: 0.390 ms	
18	Execution Time: 0.136 ms	

Before indexing, the cost was 33.78..36.28 with an execution time of 0.145 ms. After indexing, the cost reduced to 16.56..16.60 and the execution time improved to 0.136 ms.

C. Retrieving total number of active users using usage_metrics and ai_platforms table

	QUERY PLAN text	
1	Sort (cost=49.91..50.41 rows=200 width=126) (actual time=0.036..0.037 rows=2 loops=1)	
2	Sort Key: (sum(um.active_users)) DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> HashAggregate (cost=40.27..42.27 rows=200 width=126) (actual time=0.027..0.027 rows=2 loops=1)	
5	Group Key: p.platform_name	
6	Batches: 1 Memory Usage: 40kB	
7	-> Hash Join (cost=22.15..37.97 rows=460 width=122) (actual time=0.022..0.023 rows=2 loops=1)	
8	Hash Cond: (um.platform_id = p.platform_id)	
9	-> Seq Scan on usage_metrics um (cost=0.00..14.60 rows=460 width=8) (actual time=0.008..0.008 rows=2 loops=1)	
10	-> Hash (cost=15.40..15.40 rows=540 width=122) (actual time=0.008..0.009 rows=2 loops=1)	
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
12	-> Seq Scan on ai_platforms p (cost=0.00..15.40 rows=540 width=122) (actual time=0.006..0.007 rows=2 loops=1)	
13	Planning Time: 0.297 ms	
14	Execution Time: 0.056 ms	

To improve performance, indexes were created on usage_metrics.platform_id and a composite index on (platform_id, active_users).

	QUERY PLAN text	
1	Sort (cost=17.44..17.44 rows=2 width=126) (actual time=0.049..0.049 rows=2 loops=1)	
2	Sort Key: (sum(um.active_users)) DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> GroupAggregate (cost=17.39..17.43 rows=2 width=126) (actual time=0.044..0.045 rows=2 loops=1)	
5	Group Key: p.platform_name	
6	-> Sort (cost=17.39..17.40 rows=2 width=122) (actual time=0.040..0.040 rows=2 loops=1)	
7	Sort Key: p.platform_name	
8	Sort Method: quicksort Memory: 25kB	
9	-> Nested Loop (cost=0.15..17.38 rows=2 width=122) (actual time=0.022..0.024 rows=2 loops=1)	
10	-> Seq Scan on usage_metrics um (cost=0.00..1.02 rows=2 width=8) (actual time=0.009..0.009 rows=2 loops=1)	
11	-> Index Scan using ai_platforms_pkey on ai_platforms p (cost=0.15..8.17 rows=1 width=122) (actual time=0.005..0.005 rows=1 loops=1)	
12	Index Cond: (platform_id = um.platform_id)	
13	Planning Time: 0.475 ms	
14	Execution Time: 0.082 ms	

Before indexing, the cost was 49.91..50.41 with an execution time of 0.056 ms; after indexing, the cost reduced to 17.44..17.74 and the execution time increased slightly to 0.082 ms, indicating better scalability with indexed joins and groupings.

XII. TRANSACTIONS AND TRIGGERS

A new query and its response are inserted together. But the response has a negative response speed, violating a constraint and failing the transaction. The failure is logged.

A transaction is implemented that attempts to insert a query and a response. The response contains a deliberately invalid response_speed_sec value of -2.5, violating a custom constraint. PostgreSQL rolls back the entire transaction upon violation, maintaining atomicity.

We simulate failure detection using a DO block and an EXCEPTION handler that logs the error into a transaction_log table. This ensures that any invalid transaction can be traced and no partial data is committed — a practical application of transaction integrity and failure recovery.

```

Query Query History
1 ALTER TABLE responses
2 ADD CONSTRAINT chk_response_speed_positive CHECK (response_speed_sec > 0);
3
4 CREATE TABLE transaction_log (
5     log_id SERIAL PRIMARY KEY,
6     action TEXT,
7     status TEXT,
8     log_time TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE OR REPLACE FUNCTION insert_query_and_response_fail_test()
12 RETURNS VOID AS $$
13 DECLARE
14     v_query_id INT;
15 BEGIN
16     INSERT INTO queries (
17         user_id, query_type_id, topic_id, device_id, language_id, input_text, input_text_length
18     ) VALUES (
19         1, 1, 1, 1, 1, 'test query', 10
20     );
21 EXCEPTION
22     WHEN CONSTRAINT_VIOLATION THEN
23         RAISE NOTICE 'Transaction failed due to constraint violation: %', SQLERRM;
24         INSERT INTO transaction_log (action, status, log_time) VALUES (
25             'Failed to insert query and response', 'FAILED', CURRENT_TIMESTAMP
26         );
27 END;
28
29 DO $$
30 BEGIN
31     insert_query_and_response_fail_test();
32 END;
33
Data Output Messages Notifications Failed Transaction
Showing rows: 1 to 1 Page No: 1
log_id (PK) integer action text status text log_time timestamp without time zone
1 3 Failed to insert response (manual speed check) FAILED 2025-04-23 20:44:24.80676

```

Next, we implement a new transaction that attempts to insert a new user query and its response. The response includes a deliberately invalid response_speed_sec, which triggers a manually raised exception. This simulates a transaction failure due to business rule violation. While PostgreSQL doesn't support native "ON TRANSACTION FAIL" triggers, we simulate one using an EXCEPTION block and a transaction_log table to log the failure event.

This ensures transactional integrity, demonstrates how PostgreSQL rolls back operations upon error, and shows how failure scenarios can be captured programmatically for audit and debugging purposes.

What happens when a transaction gets aborted? When a transaction aborts, all changes inside the transaction are rolled back. Nothing inside the transaction is permanently saved, except if something happened outside the transaction (like our log insert). The database is left exactly as it was before the BEGIN command — no partial changes.

```

Query Query History
1 SELECT * FROM usage_metrics
2
3 SELECT metric_id FROM usage_metrics ORDER BY metric_id DESC LIMIT 1;
4
5 BEGIN;
6
7 -- Lock the row
8 SELECT * FROM usage_metrics WHERE metric_id = 1 FOR UPDATE;
9
10 -- Delay
11 UPDATE usage_metrics
12 SET active_users = active_users + 50
13 WHERE metric_id = 1;
14
15 ROLLBACK
16 SELECT * FROM usage_metrics
17
18
Data Output Messages Notifications
ROLLBACK
Query returned successfully in 27 msec.

```

```

Query Query History
1 BEGIN;
2
3 UPDATE usage_metrics
4 SET active_users = active_users + 1000
5 WHERE metric_id = 1;
6 COMMIT
7 SELECT * FROM usage_metrics
8
Data Output Messages Notifications COMMIT
Showing rows: 1 to 2 Page No: 1 of 1
metric_id (PK) integer date month_name integer weekday character varying (20) platform_id integer model_id integer active_users integer new_users integer channel_users integer daily_churn_rate numeric (5,2)
1 2 2025-04-20 4 Sunday 1 1 1200 350 80 6.70
2 1 2025-04-20 4 Sunday 1 1 3102 300 100 10.00

```

This is important to keep ACID properties — especially Atomicity and Consistency. In our implementation, a trigger pre-validated the input and safely logged failures separately before throwing an exception, demonstrating safe error handling in a transaction.

TEAM CONTRIBUTION

TABLE II. INDIVIDUAL CONTRIBUTION

Anirudh	Suvidha	Vidit
33.33 %	33.33%	33.33%

ACKNOWLEDGMENT

We sincerely thank Dr. Shamsad Parvin for her guidance and support throughout this project. We also appreciate Mr. Pengyu Yan for his valuable assistance and feedback, which greatly contributed to our learning and project development.

REFERENCES

- [1] A. Khan, *DeepSeek vs ChatGPT: AI Platform Comparison*, Kaggle, 2025. [Online]. Available: <https://kaggle.com/datasets/khanaakif/deepseek-vs-chatgpt-ai-platform-comparison/data>. [Accessed: 16-Mar-2025].